

Slide 1

Overview OBJECT ORIENTED ANALYSIS AND DESIGN

Slide 2

UNIT – I Introduction to UML: Importance of modeling, principles of modeling, object oriented modeling, Conceptual model of the UML, Architecture, and Software Development Life Cycle.

Slide 3

UNIT – I The UML is a graphical language for visualizing, specifying, constructing, and documenting the artifacts of a software-intensive system. The UML gives you a standard way to write a system's blueprints, covering conceptual things, such as business processes and system functions, as well as concrete things, such as classes written in a specific programming language, database schemas, and reusable software components.

Slide 4

Importance of modeling A model is a simplification of reality. A model provides the blueprints of a system. A model may be structural, emphasizing the organization of the system, or it may be behavioral, emphasizing the dynamics of the system.

Slide 5

Aims of modeling We build models so that we can better understand the system we are developing. Through modeling, we achieve four aims. 1. Models help us to visualize a system as it is or as we want it to be. 2. Models permit us to specify the structure or behavior of a system. 3. Models give us a template that guides us in constructing a system.

Slide 6

Aims of modeling (Cont.) 4. Models document the decisions we have made. We build models of complex systems because we cannot comprehend such a system in its entirety.

Slide 7

Principles of Modeling There are four basic principles of model 1. The choice of what models to create has a profound influence on how a problem is attacked and how a solution is shaped. 2. Every model may be expressed at different levels of precision. 3. The best models are connected to reality.

Slide 8

Principles of Modeling (Cont.) 4. No single model is sufficient. Every nontrivial system is best approached through a small set of nearly independent models. Object Oriented Modeling In software, there are several ways to approach a model. The two most common ways are 1. Algorithmic perspective

Slide 9

Principles of Modeling (Cont.) 2. Object-oriented perspective

Slide 10

Algorithmic Perspective • The traditional view of software development takes an algorithmic perspective. • In this approach, the main building block of all software is the procedure or - 1 of 15 -- function. • This view leads developers to focus on issues of control and the decomposition of

Slide 11

Algorithmic Perspective (Cont.) larger algorithms into smaller ones. ð. As requirements change and the system grows, systems built with an algorithmic focus turn out to be very hard to maintain.

Slide 12

Object-Oriented Perspective ð· The contemporary view of software development takes an object-oriented perspective. ð· In this approach, the main building block of all software systems is the object or class. ð· A class is a description of a set of common objects.

Slide 13

Object-Oriented Perspective (Cont.) • Every object has identity, state, and behavior. •

Object-oriented development provides the conceptual foundation for assembling systems out of components using technology such as Java Beans or COM+.

Slide 14

An Overview of UML ð· The Unified Modeling Language is a standard language for writing software blueprints. The UML may be used to visualize, specify, construct, and document the artifacts of a software-intensive system. ð· The UML is appropriate for modeling systems ranging from enterprise information systems to distributed Web-based applications and even to hard real time

Slide 15

An Overview of UML (Cont.) embedded systems. It is a very expressive language, addressing all the views needed to develop and then deploy such systems. The UML is a language for

- Visualizing
- Specifying

Slide 16

An Overview of UML (Cont.) • Constructing • Documenting • Visualizing The UML is more than just a bunch of graphical symbols. Rather, behind each symbol in the UML notation is a well-defined semantics. In this manner, one developer can write a model in the UML, and another developer, or even another tool,

Slide 17

An Overview of UML (Cont.) can interpret that model unambiguously ð· Specifying means building models that are precise, unambiguous, and complete. ð· Constructing the UML is not a visual programming language, but its models can be directly connected to a variety of programming languages ð· Documenting a healthy software organization produces all sorts of artifacts in addition

Slide 18

An Overview of UML (Cont.) to raw executable code. These artifacts include o
Requirements

Slide 19

o Architecture o Design o Source code o Project plans o Tests o Prototypes

Slide 20

o Architecture (Cont.) o Releases - 2 of 15 --

Slide 21

Conceptual model of UML To understand the UML, you need to form a conceptual model of the language, and this requires learning three major elements: ð· Basic building blocks of the UML ð· Rules ð· Common Mechanisms in the UML

Slide 22

Conceptual model of UML (Cont.) Basic building blocks of the UML: Vocabulary of the UML can be defined 1. Things

Slide 23

2. Relationships

Slide 24

3. Diagrams

Slide 25

Things in the UML There are four kinds of things in the UML: • Structural things • Behavioral things • Grouping things • Annotational things

Slide 26

Things in the UML (Cont.) Structural things are the nouns of UML models. These are the mostly static parts of a model, representing elements that are either conceptual or physical. In all, there are seven kinds of structural things. 1. Classes 2. Interfaces

Slide 27

Things in the UML (Cont.) 3. Collaborations 4. Use cases 5. Active classes 6. Components 7. Nodes

Slide 28

Class • It is a description of a set of objects that share the same attributes, operations, relationships, and semantics. • A class implements one or more interfaces. • Graphically, a class is rendered as a rectangle, usually including its name, attributes, and operations.

Slide 29

Class (Cont.) . Fig: Class - 3 of 15 --

Slide 30

Interface

- Interface is a collection of operations that specify a service of a class or component.
- An interface therefore describes the externally visible behavior of that element.
- An interface might represent the complete behavior of a class or component or only a part of that behavior.

Slide 31

Interface (Cont.) • An interface is rendered as a circle together with its name. An interface rarely stands alone. Rather, it is typically attached to the class or component that realizes the interface. IUnknown Fig: Interface

Slide 32

Collaboration • It defines an interaction and is a society of roles and other elements that work together to provide some cooperative behavior that's bigger than the sum of all the elements. • Therefore, collaborations have structural, as well as behavioral, dimensions. • A given class might participate in several collaborations.

Slide 33

Collaboration (Cont.) • Graphically, a collaboration is rendered as an ellipse with dashed lines, usually including only its name. Fig: Collaboration

Slide 34

Use case ð· Use case is a description of set of sequence of actions that a system performs that yields an observable result of value to a particular actor ð· Use case is used to structure the behavioral things in a model. ð· A use case is realized by a collaboration. Graphically, a use case is rendered as an ellipse with solid lines, usually including only its name

Slide 35

Use case (Cont.) Fig: Use case Active class • It is just like a class except that its objects represent elements whose behavior is concurrent with other elements. • Graphically, an active class is rendered just like a class, but with heavy lines,

Slide 36

Use case (Cont.) usually including its name, attributes, and operations. - 4 of 15 -- Fig:
Active class

Slide 37

Component δ . It is a physical and replaceable part of a system that conforms to and provides the realization of a set of interfaces. δ . Graphically, a component is rendered as a rectangle with tabs. Fig: Component

Slide 38

Node • It is a physical element that exists at run time and represents a computational resource, generally having at least some memory and, often, processing capability. • Graphically, a node is rendered as a cube, usually including only its name. Fig: Node Behavioral Things

Slide 39

Node (Cont.) • They are the dynamic parts of UML models. These are the verbs of a model, representing behavior over time and space. • In all, there are two primary kinds of behavioral things. Interaction State machine

Slide 40

Node (Cont.) Interaction ð Interaction is a behavior that comprises a set of messages exchanged among a set of objects within a particular context to accomplish a specific purpose ð An interaction involves a number of other elements, including messages, action sequences and links

Slide 41

Node (Cont.) • Graphically a message is rendered as a directed line, almost always including the name of its operation Fig: Message State Machine • State machine is a behavior that specifies the sequences of states an object or an

Slide 42

Node (Cont.) interaction goes through during its lifetime in response to events, together with its responses to those events - 5 of 15 -- ð· State machine involves a number of other elements, including states, transitions, events and activities

Slide 43

Node (Cont.) • Graphically, a state is rendered as a rounded rectangle, usually including its name and its substates Fig: State machine Grouping Things 1. These are the organizational parts of UML models. These are the boxes into

Slide 44

Node (Cont.) which a model can be decomposed 2. There is one primary kind of grouping thing, namely, packages. Package 3. A package is a general-purpose mechanism for organizing elements into groups. Structural things, behavioral things, and even other grouping things may be placed in

Slide 45

Node (Cont.) a package • Graphically, a package is rendered as a tabbed folder, usually including only its name and, sometimes, its contents. Fig: Package Annotational things

Slide 46

Node (Cont.) • These are the explanatory parts of UML models. These are the comments you may apply to describe about any element in a model. • There is one primary kind of Annotational thing, namely, note. Note • A note is simply a symbol for rendering constraints and comments attached to an

Slide 47

Node (Cont.) element or a collection of elements. • Graphically, a note is rendered as a rectangle with a dog-eared corner, together with a textual or graphical comment. Fig:

Note Relationships in the UML: There are four kinds of relationships in the UML:

Slide 48

Node (Cont.) 1. Dependency 2. Association 3. Generalization 4. Realization - 6 of 15 --

Slide 49

Node (Cont.) Dependency ð Dependency is a semantic relationship between two things in which a change to one thing may affect the semantics of the other thing ð Graphically a dependency is rendered as a dashed line, possibly directed, and occasionally including a label.

Slide 50

Node (Cont.) Fig: Dependency Association • Association is a structural relationship that describes a set of links, a link being a connection among objects. • Graphically an association is rendered as a solid line, possibly directed, occasionally

Slide 51

Node (Cont.) including a label, and often containing other adornments, such as multiplicity and role names. Fig: Association Generalization Generalization is a relationship that implements the concept of inheritance, a fundamental principle

Slide 52

Node (Cont.) of object-oriented programming. This relationship occurs between two entities or objects, where one entity is the parent (or superclass) and the other is the child (or subclass). The child inherits the attributes, operations, and relationships of the parent, allowing for reuse and extension of existing functionality. Fig: Generalization

Slide 53

Node (Cont.) Realization

- Realization is a semantic relationship between classifiers, wherein one classifier specifies a contract that another classifier guarantees to carry out.
- Graphically a realization relationship is rendered as a cross between a generalization and a dependency relationship.

Slide 54

Node (Cont.) Fig: Realization

Slide 55

Diagrams in the UML • Diagram is the graphical presentation of a set of elements, most often rendered as a connected graph of vertices (things) and arcs (relationships). • In theory, a diagram may contain any combination of things and relationships. • For this reason, the UML includes nine such diagrams - 7 of 15 --

Slide 56

Diagrams in the UML (Cont.) Class diagram • Class diagram • Object diagram • Use case diagram • Sequence diagram

Slide 57

Diagrams in the UML (Cont.) ð Collaboration diagram ð State chart diagram ð Activity diagram ð Component diagram ð Deployment diagram

Slide 58

Diagrams in the UML (Cont.) ð· A class diagram shows a set of classes, interfaces, and collaborations and their relationships. ð· Class diagrams that include active classes address the static process view of a system. Object diagram ð· Object diagrams represent static snapshots of instances of the things found in

Slide 59

Diagrams in the UML (Cont.) class diagrams • These diagrams address the static design view or static process view of a system • An object diagram shows a set of objects and their relationships Use case diagram • A use case diagram shows a set of use cases and actors and their relationships

Slide 60

Diagrams in the UML (Cont.) ð· Use case diagrams address the static use case view of a system. ð· These diagrams are especially important in organizing and modeling the behaviors of a system.

Slide 61

Interaction Diagrams • Both sequence diagrams and collaboration diagrams are kinds of interaction diagrams • Interaction diagrams address the dynamic view of a system • A sequence diagram is an interaction diagram that emphasizes the time-ordering of messages

Slide 62

Interaction Diagrams (Cont.) • A collaboration diagram is an interaction diagram that emphasizes the structural organization of the objects that send and receive messages • Sequence diagrams and collaboration diagrams are isomorphic, meaning that you can take one and transform it into the other. Statechart diagram

Slide 63

Interaction Diagrams (Cont.) • A state chart diagram shows a state machine, consisting of states, transitions, events, and activities • State chart diagrams address the dynamic view of a system • They are especially important in modeling the behavior of an interface, class, or collaboration and emphasize the event-ordered behavior of an object.

Slide 64

Interaction Diagrams (Cont.) Activity diagram An activity diagram is a special kind of a state chart diagram that shows the flow from activity to activity within a system Activity diagrams address the dynamic view of a system They are especially important in modeling the function of a system and

Slide 65

Interaction Diagrams (Cont.) emphasize the flow of control among objects. - 8 of 15 --

Component diagram ð. A component diagram shows the organizations and dependencies among a set of components.

Slide 66

Interaction Diagrams (Cont.) ð Component diagrams address the static implementation view of a system ð They are related to class diagrams in that a component typically maps to one or more classes, interfaces, or collaborations. Deployment diagram ð A deployment diagram shows the configuration of run-time processing nodes and

Slide 67

Interaction Diagrams (Cont.) the components that live on them • Deployment diagrams address the static deployment view of an architecture

Slide 68

Rules of the UML The UML has semantic rules for 1. Names What you can call things, relationships, and diagrams 2. Scope The context that gives specific meaning to a name 3. Visibility How those names can be seen and used by others 4. Integrity How things properly and consistently relate to one another

Slide 69

Rules of the UML (Cont.) 5. Execution What it means to run or simulate a dynamic model Models built during the development of a software-intensive system tend to evolve and may be viewed by many stakeholders in different ways and at different times. For this reason, it is common for the development team to not only build models that are well- formed, but also to build models that are

Slide 70

Rules of the UML (Cont.) 1. Elided Certain elements are hidden to simplify the view 2. Incomplete Certain elements may be missing 3. Inconsistent The integrity of the model is not guaranteed Common Mechanisms in the UML UML is made simpler by the presence of four common mechanisms that apply consistently

Slide 71

Rules of the UML (Cont.) throughout the language. 1. Specifications 2. Adornments 3. Common divisions 4. Extensibility mechanisms

Slide 72

Rules of the UML (Cont.) Specification

- They provides a textual statement of the syntax and semantics of that building block.
- The UML's specifications provide a semantic backplane that contains all the parts of all the models of a system, each part related to one another in a consistent fashion

Slide 73

Rules of the UML (Cont.) Adornments

- Most elements in the UML have a unique and direct graphical notation that provides a visual representation of the most important aspects of the element.
- A class's specification may include other details, such as whether it is abstract or the visibility of its attributes and operations.

Slide 74

Rules of the UML (Cont.) - 9 of 15 -- • Many of these details can be rendered as graphical or textual adornments to the class's basic rectangular notation.

Common Divisions In modeling object-oriented systems, the world often gets divided in at least a couple of

Slide 75

Rules of the UML (Cont.) ways. First, there is the division of class and object. A class is an abstraction; an object is one concrete manifestation of that abstraction. In the UML, you can model classes as well as objects, as shown in Figure 2-17 In this figure, there is one class, named Customer, together with three objects: Jan (which

Slide 76

Rules of the UML (Cont.) is marked explicitly as being a Customer object), :Customer (an anonymous Customer object), and Elyse (which in its specification is marked as being a kind of Customer object, although it's not shown explicitly here). Second, there is the separation of interface and implementation. An interface declares a contract, and an implementation represents one concrete realization of that contract,

Slide 77

Rules of the UML (Cont.) responsible for faithfully carrying out the interface's complete semantics. In the UML, you can model both interfaces and their implementations, as shown in Figure 2-18. In this figure, there is one component named spellingwizard.dll that implements two interfaces, IUnknown and ISpelling. - 10 of 15 --

Slide 78

Rules of the UML (Cont.) Extensibility Mechanisms The UML's extensibility mechanisms include 1. Stereotypes 2. Tagged values 3. Constraints

Slide 79

Rules of the UML (Cont.) ð Stereotype extends the vocabulary of the UML, allowing you to create new kinds of building blocks that are derived from existing ones but that are specific to your problem ð A tagged value extends the properties of a UML building block, allowing you to create new information in that element's specification ð A constraint extends the semantics of a UML building block, allowing you to add new

Slide 80

Rules of the UML (Cont.) rules or modify existing ones. Examples for these are

Slide 81

Architecture • A system's architecture is perhaps the most important artifact that can be used to manage these different viewpoints and so control the iterative and incremental development of a system throughout its life cycle. • Architecture is the set of significant decisions about • The organization of a software system

Slide 82

Architecture (Cont.) ð· The selection of the structural elements and their interfaces by which the system is composed ð· Their behavior, as specified in the collaborations among those elements ð· The composition of these structural and behavioral elements into progressively larger subsystems

Slide 83

Architecture (Cont.) ð· The architectural style that guides this organization: the static and dynamic elements and their interfaces, their collaborations, and their composition. ð· Software architecture is not only concerned with structure and behavior, but also with usage, functionality, performance, resilience, reuse, comprehensibility, economic and technology constraints and trade-offs, and aesthetic concerns.

Slide 84

Architecture (Cont.) - 11 of 15 --

Slide 85

Fig: Modeling a System's Architecture Use case view • The use case view of a system encompasses the use cases that describe the behavior of the system as seen by its end users, analysts, and testers. • With the UML, the static aspects of this view are captured in use case diagrams • The dynamic aspects of this view are captured in interaction diagrams, state chart

Slide 86

Fig: Modeling a System's Architecture (Cont.) diagrams, and activity diagrams.

Design View • The design view of a system encompasses the classes, interfaces, and collaborations that form the vocabulary of the problem and its solution. • This view primarily supports the functional requirements of the system, meaning the

Slide 87

Fig: Modeling a System's Architecture (Cont.) services that the system should provide to its end users. Process View • The process view of a system encompasses the threads and processes that form the system's concurrency and synchronization mechanisms. • This view primarily addresses the performance, scalability, and throughput of the

Slide 88

Fig: Modeling a System's Architecture (Cont.) system. Implementation View ð The implementation view of a system encompasses the components and files that are used to assemble and release the physical system. ð This view primarily addresses the configuration management of the system's releases,

Slide 89

Fig: Modeling a System's Architecture (Cont.) made up of somewhat independent components and files that can be assembled in various ways to produce a running system. Deployment View • The deployment view of a system encompasses the nodes that form the system's hardware topology on which the system executes.

Slide 90

Fig: Modeling a System's Architecture (Cont.) • This view primarily addresses the distribution, delivery, and installation of the parts that make up the physical system. - 12 of 15 -- • With the UML, the static aspects of this view are captured in deployment diagrams; the dynamic aspects of this view are captured in interaction diagrams, statechart

Slide 91

Fig: Modeling a System's Architecture (Cont.) diagrams, and activity diagrams. ð Each of these five views can stand alone so that different stakeholders can focus on the issues of the system's architecture that most concern them. ð These five views also interact with one another• nodes in the deployment view hold components in the implementation view that, in turn, represent the physical realization

Slide 92

Fig: Modeling a System's Architecture (Cont.) of classes, interfaces, collaborations, and active classes from the design and process views. • The UML permits you to express every one of these five views and their interactions.

Slide 93

Software Development Life Cycle ð The UML is largely process-independent, meaning that it is not tied to any particular software development life cycle. ð However, to get the most benefit from the UML, you should consider a process that is ð Use case driven

Slide 94

• Architecture-centric • Iterative and incremental Use case driven • It means that use cases are used as a primary artifact for establishing the desired behavior of the system, for verifying and validating the system's architecture, for testing, and for communicating among the stakeholders of the project.

Slide 95

Architecture-centric ð It means that a system's architecture is used as a primary artifact for Conceptualizing, constructing, managing, and evolving the system under development.ð An iterative process ð It is one that involves managing a stream of executable releases.ð

Slide 96

Architecture-centric (Cont.) An incremental process • It is one that involves the continuous integration of the system's architecture to produce these releases, with each new release embodying incremental improvements over the other. • Together, an iterative and incremental process is risk-driven, meaning that each

Slide 97

Architecture-centric (Cont.) new release is focused on attacking and reducing the most significant risks to the success of the project. This use case driven, architecture-centric, and iterative/incremental process can be broken into phases.đ đ· A phase is the span of time between two major milestones of the process, when a well-defined set of objectives are met, artifacts are completed, and decisions are

Slide 98

Architecture-centric (Cont.) made whether to move into the next phase. As below figure shows, there are four phases in the software development life cycle: inception, elaboration, construction, and transition. In the diagram, 13 of 15 -- workflows are plotted against these phases, showing their varying degrees of

Slide 99

Architecture-centric (Cont.) focus over time.

Slide 100

Fig: Figure Software Development Life Cycle Inception phase: ð· It is the first phase of the processð ð· When the seed idea for the development is brought up to the point of being• at least internally• sufficiently well-founded to warrant entering into the elaboration phase.ð

Elaboration phase:

Slide 101

Fig: Figure Software Development Life Cycle (Cont.)

- It is the second phase of the process
- When the product vision and its architecture are defined.
- In this phase, the system's requirements are articulated, prioritized, and baselined.
- A system's requirements may range from general vision statements to precise evaluation criteria, each specifying particular functional or nonfunctional behavior

Slide 102

Fig: Figure Software Development Life Cycle (Cont.) and each providing a basis for testing.đ Construction phase: đ· It is the third phase of the process.đ đ· When the software is brought from an executable architectural baseline to being ready to be transitioned to the user community.đ

Slide 103

Fig: Figure Software Development Life Cycle (Cont.) ð· The system's requirements and especially its evaluation criteria are constantly reexamined against the business needs of the project, and resources are allocated as appropriate to actively attack risks to the project.ð Transition phase: ð· It is the fourth phase of the process.ð

Slide 104

Fig: Figure Software Development Life Cycle (Cont.)
• When the software is turned into the hands of the user community.
• - 14 of 15 --
• The software development process end here, for even during this phase, the system is continuously improved, bugs are eradicated, and features that didn't make an earlier release are added.

Slide 105

Fig: Figure Software Development Life Cycle (Cont.)

- One element that distinguishes this process and that cuts across all four phases is an iteration.
- An iteration is a distinct set of activities, with a baselined plan and evaluation criteria that result in a release, either internal or external.

Slide 106

Fig: Figure Software Development Life Cycle (Cont.) ð· This means that the software development life cycle can be characterized as involving a continuous stream of executable releases of the system's architecture.ð It is this emphasis on architecture as an important artifact that drives the UML to focus on modeling the different views of a

Slide 107

Fig: Figure Software Development Life Cycle (Cont.) - 15 of 15 --